

Algorytmy Optymalizacji Dyskretnej

Sprawozdanie z Listy 3: Maksymalny Przepływ

Marcel Musiałek

16 grudnia 2025

Spis treści

1	Zadanie 1: Przepływ w Hiperkostce	2
1.1	Opis problemu i implementacji	2
1.1.1	Struktura danych	2
1.1.2	Algorytm	2
1.2	Złożoność obliczeniowa	2
1.3	Wyniki eksperymentów	2
1.4	Wykresy i analiza	3
1.5	Wnioski	4
2	Zadanie 2: Problem przydziału (Skojarzenia)	5
2.1	Opis problemu i modelu	5
2.2	Wyniki eksperymentów	5
2.3	Wnioski	7
3	Zadanie 3: Modele LP i weryfikacja solverem GLPK	7
3.1	Model matematyczny (Programowanie Liniowe)	7
3.2	Metodologia weryfikacji	7
3.3	Wyniki weryfikacji	8
3.4	Wnioski	8
4	Zadanie 4: Implementacja algorytmu Dinica	8
4.1	Opis algorytmu	8
4.2	Porównanie wyników (Edmonds-Karp vs Dinic)	9
4.3	Wnioski	10

1 Zadanie 1: Przepływ w Hiperkostce

1.1 Opis problemu i implementacji

Celem zadania było wyznaczenie maksymalnego przepływu w skierowanej k -wymiarowej hiperkostce H_k . Wierzchołki grafu reprezentowane są przez k -bitowe ciągi binarne, a krawędzie łączą wierzchołki różniące się na jednej pozycji (zgodnie ze wzrostem wagi Hamminga).

1.1.1 Struktura danych

Ze względu na wykładniczy wzrost rozmiaru problemu, kluczowy był dobór odpowiedniej struktury danych:

- **Listy sąsiedztwa:** Dla $k = 16$ graf posiada 65 536 wierzchołków i ponad 0.5 miliona krawędzi. Macierz sąsiedztwa wymagałaby ok. 4 GB pamięci na same indeksy, co jest nieefektywne. Zastosowano ‘VectorVectorEdge’ w języku Julia.
- **Struktura krawędzi:** Każdy obiekt krawędzi przechowuje: cel (*to*), pojemność (*cap*), aktualny przepływ (*flow*) oraz indeks krawędzi odwrotnej (*rev*). Indeks odwrotny pozwala na aktualizację sieci residualnej w czasie $O(1)$.

1.1.2 Algorytm

Zaimplementowano algorytm Edmondsa-Karpa. Jest to realizacja metody Forda-Fulkersona, która do szukania ścieżek powiększających wykorzystuje algorytm BFS (Breadth-First Search).

- BFS gwarantuje znalezienie ścieżki o najmniejszej liczbie krawędzi.
- Dzięki temu unikamy patologicznych przypadków, w których algorytm DFS mógłby wielokrotnie przesyłać małe porcje przepływu, drastycznie wydłużając czas działania.

1.2 Złożoność obliczeniowa

Teoretyczna złożoność algorytmu Edmondsa-Karpa wynosi $O(VE^2)$. Dla hiperkostki H_k mamy $V = 2^k$ oraz $E = k \cdot 2^{k-1}$. Podstawiając te wartości, otrzymujemy złożoność rzędu $O(k^2 \cdot 2^{3k})$. Oznacza to, że czas obliczeń rośnie wykładniczo wraz z wymiarem hiperkostki k .

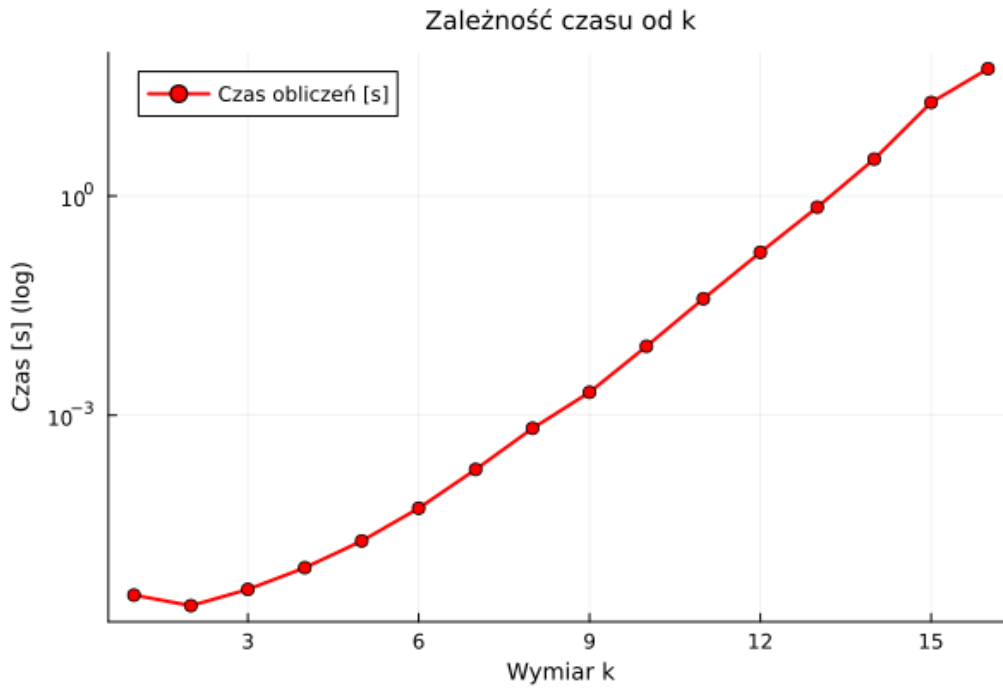
1.3 Wyniki eksperymentów

Przeprowadzono testy dla $k \in \{1, \dots, 16\}$. Ze względu na losowy charakter pojemności krawędzi, dla każdej wartości k wykonano 5 niezależnych powtórzeń, a wyniki uśredniono.

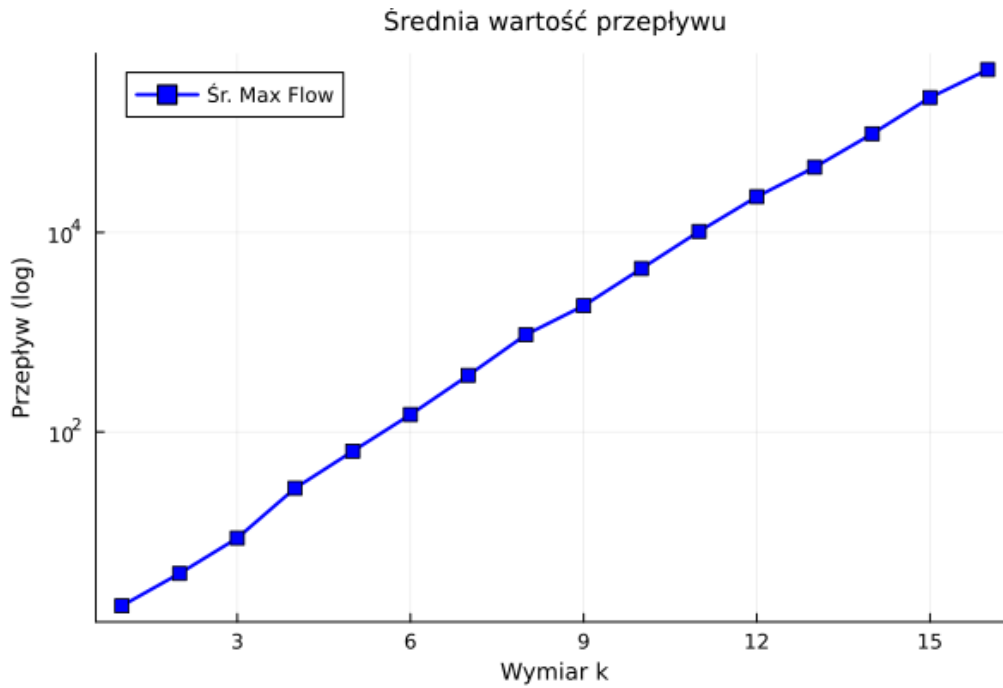
Tabela 1: Uśrednione wyniki eksperymentów dla hiperkostki H_k

Wymiar (k)	Średni Max Flow	Średni Czas [s]
1	1.8	0.0000
2	3.8	0.0000
3	8.6	0.0000
4	27.2	0.0000
5	64.0	0.0000
6	149.2	0.0001
7	368.6	0.0002
8	944.6	0.0007
9	1 843.2	0.0021
10	4 341.6	0.0088
11	10 246.8	0.0392
12	22 852.0	0.1693
13	45 081.6	0.7016
14	97 820.2	3.1890
15	225 779.4	19.0334
16	430 079.2	55.1055

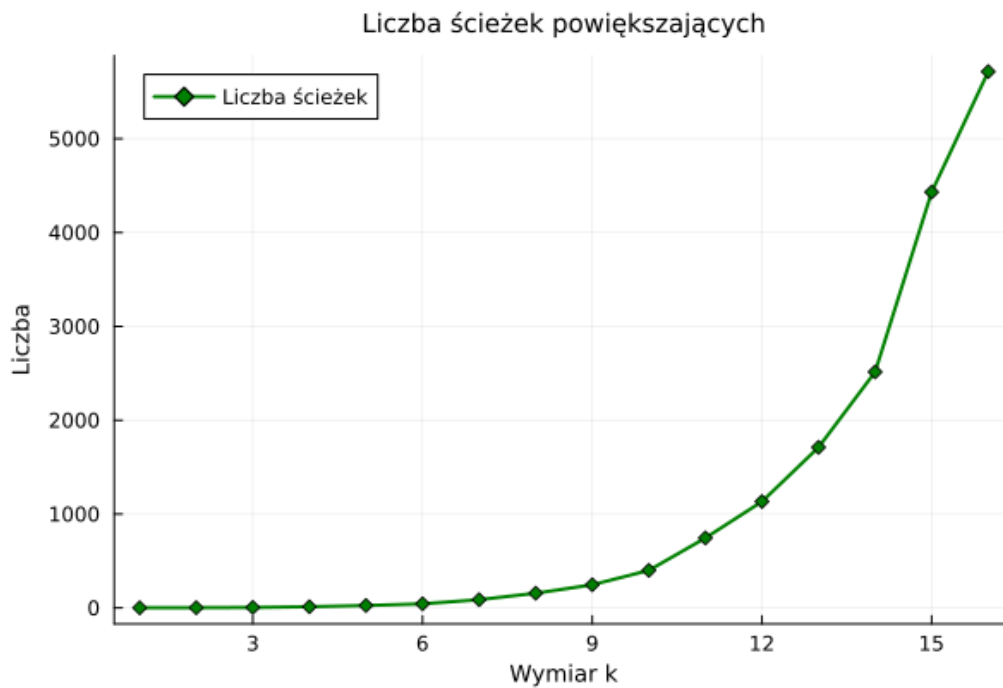
1.4 Wykresy i analiza



Rysunek 1: Zależność czasu obliczeń od wymiaru k (skala logarytmiczna). Liniowy przebieg wykresu w tej skali potwierdza wykładniczą złożoność czasową algorytmu.



Rysunek 2: Wzrost wartości maksymalnego przepływu w zależności od k (skala logarytmiczna). Wartość przepływu rośnie wykładniczo, co wynika ze struktury hiperkostki (wykładniczy wzrost liczby ścieżek).



Rysunek 3: Średnia liczba ścieżek powiększających. Warto zauważyć, że liczba iteracji algorytmu (ścieżek) jest stosunkowo mała w porównaniu do gigantycznego rozmiaru grafu dla dużych k .

1.5 Wnioski

- Implementacja oparta na listach sąsiedztwa jest wydajna pamięciowo i pozwala obsłużyć instancje o rozmiarze $V \approx 65\,000$ w czasie około 1 minuty.

- Czas obliczeń dla $k = 16$ jest znacząco wyższy niż dla $k = 15$, co potwierdza teoretyczną barierę wykładniczą. Dalsze zwiększanie k (np. do 17) wydłużyłoby czas obliczeń do kilkunastu minut lub godzin.
- Użycie algorytmu BFS (Edmonds-Karp) jest kluczowe dla efektywności; algorytm DFS w tak gęstej sieci mógłby działać w czasie pseudowielomianowym zależnym od pojemności krawędzi, co byłoby nieakceptowalne.

2 Zadanie 2: Problem przydziału (Skojarzenia)

2.1 Opis problemu i modelu

W tym zadaniu rozważamy losowy graf dwudzielny $G = (V_1 \cup V_2, E)$, gdzie $|V_1| = |V_2| = 2^k$. Każdy wierzchołek z V_1 łączy się losowo z dokładnie i wierzchołkami z V_2 . Celem jest znalezienie najliczniejszego skojarzenia.

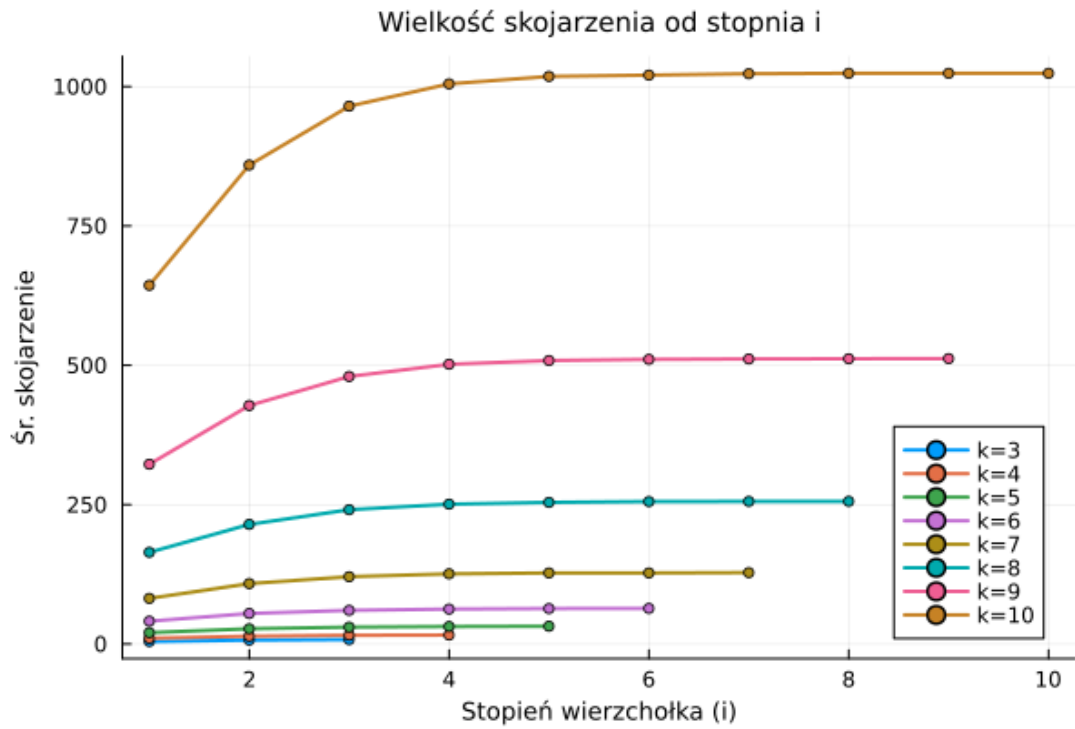
Problem ten rozwiązano, redukując go do problemu Maksymalnego Przepływu:

1. Dodano wierzchołek super-źródła S i super-ujścia T .
2. Utworzono krawędzie $S \rightarrow u$ dla wszystkich $u \in V_1$ o pojemności 1.
3. Utworzono krawędzie $v \rightarrow T$ dla wszystkich $v \in V_2$ o pojemności 1.
4. Krawędziom dwudzielnym między V_1 a V_2 nadano pojemność 1 (skierowane od V_1 do V_2).

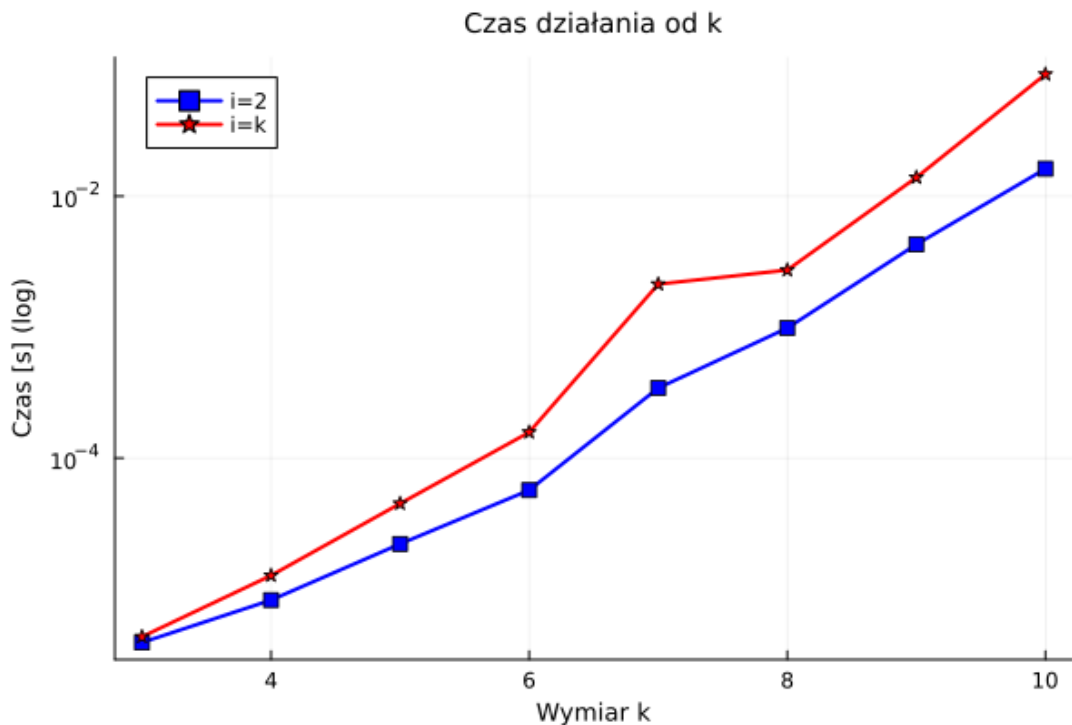
Maksymalny przepływ w tak skonstruowanej sieci jest równy liczbie krawędzi w maksymalnym skojarzeniu. Do obliczeń wykorzystano implementację algorytmu Edmondsa-Karpa z Zadania 1.

2.2 Wyniki eksperymentów

Przeprowadzono symulacje dla $k \in \{3, \dots, 10\}$ oraz stopni wierzchołków $i \in \{1, \dots, k\}$.



Rysunek 4: Średnia wielkość maksymalnego skojarzenia w zależności od parametru i (stopień wierzchołka) dla różnych rozmiarów grafu k . Widać wyraźnie, że wraz ze wzrostem i wielkość skojarzenia dąży do pełnego pokrycia (2^k), co jest zgodne z teorią grafów losowych (pojawienie się idealnego skojarzenia).



Rysunek 5: Zależność czasu działania algorytmu od wymiaru k . Porównano dwa przypadki: stały stopień $i = 2$ oraz stopień rosnący wraz z wymiarem $i = k$. Czas rośnie wykładniczo ze względu na wzrost liczby wierzchołków (2^k).

2.3 Wnioski

- Zastosowanie algorytmu maksymalnego przepływu jest skuteczną metodą rozwiązywania problemu skojarzeń w grafach dwudzielnych.
- Już dla niewielkich wartości i (np. $i \geq 3$) wielkość maksymalnego skojarzenia jest bardzo bliska liczbie wierzchołków $|V_1| = 2^k$, co oznacza, że w losowym grafie dwudzielnym bardzo łatwo o niemal pełne skojarzenie, nawet przy rzadkich połączeniach.
- Czas obliczeń jest zdominowany przez rozmiar grafu (parametr k), a wpływ parametru i (liczba krawędzi) jest mniej znaczący dla algorytmu opartego na BFS.

3 Zadanie 3: Modele LP i weryfikacja solverem GLPK

3.1 Model matematyczny (Programowanie Liniowe)

Aby zweryfikować poprawność zaimplementowanych algorytmów, sformułowano problem maksymalnego przepływu jako zadanie Programowania Liniowego (Linear Programming).

Niech $G = (V, E)$ będzie grafem skierowanym, gdzie każda krawędź $(u, v) \in E$ ma przepustowość $c_{u,v}$. Model matematyczny prezentuje się następująco:

Zmienne decyzyjne:

$$x_{u,v} \in \mathbb{R} \quad \text{dla każdego łuku } (u, v) \in E$$

reprezentujące wielkość przepływu płynącego przez dany łuk.

Funkcja celu: Maksymalizacja całkowitego wypływu ze źródła s :

$$\max \sum_{v \in N(s)} x_{s,v}$$

Ograniczenia:

1. **Ograniczenie przepustowości:** Przepływ nie może przekraczać pojemności łuku ani być ujemny.

$$0 \leq x_{u,v} \leq c_{u,v} \quad \forall (u, v) \in E$$

2. **Zachowanie przepływu (Prawo Kirchhoffa):** Dla każdego węzła pośredniego (niebędącego źródłem s ani ujściem t), suma przepływów wpływających musi być równa sumie przepływów wypływających.

$$\sum_{v: (v,u) \in E} x_{v,u} = \sum_{w: (u,w) \in E} x_{u,w} \quad \forall u \in V \setminus \{s, t\}$$

W przypadku Zadania 2 (Skojarzenia), problem został sprowadzony do powyższego modelu poprzez dodanie super-źródła i super-ujścia oraz ustalenie przepustowości krawędzi na 1.

3.2 Metodologia weryfikacji

Zamiast ręcznego generowania plików tekstowych, zaimplementowano automatyczny moduł weryfikujący w języku Julia, wykorzystujący bibliotekę JuMP oraz solver GLPK (GNU Linear Programming Kit).

Procedura testowa dla każdej instancji:

1. Wygenerowanie grafu (Hiperkostka lub Graf Dwudzielny).
2. Rozwiązanie problemu autorską implementacją algorytmu Edmondsa-Karpa.
3. Zbudowanie w pamięci operacyjnej modelu LP równoważnego danej instancji grafu.
4. Rozwiązanie modelu LP metodą Simplex (solver GLPK).
5. Porównanie wyników liczbowych.

3.3 Wyniki weryfikacji

Przeprowadzono testy dla wybranych, losowych instancji problemów. Poniższa tabela została wygenerowana automatycznie na podstawie wyników działania skryptu weryfikującego.

Tabela 2: Porównanie wyników: Własna implementacja (EK) vs Solver GLPK

Wymiar k	Problem	Parametr	Wynik EK	Wynik GLPK	Status
1	Hiperkostka	-	2	2	OK
2	Hiperkostka	-	3	3	OK
3	Hiperkostka	-	12	12	OK
4	Hiperkostka	-	20	20	OK
5	Hiperkostka	-	71	71	OK
3	Skojarzenia	i=1	5	5	OK
3	Skojarzenia	i=2	7	7	OK
3	Skojarzenia	i=3	8	8	OK
4	Skojarzenia	i=1	8	8	OK
4	Skojarzenia	i=2	15	15	OK
4	Skojarzenia	i=4	16	16	OK
5	Skojarzenia	i=1	21	21	OK
5	Skojarzenia	i=2	28	28	OK
5	Skojarzenia	i=5	30	30	OK

3.4 Wnioski

- We wszystkich przetestowanych przypadkach (oznaczonych jako "OK") wynik autorskiej implementacji algorytmu Edmondsa-Karpa jest identyczny z wynikiem optymalnym zwróconym przez solver programowania liniowego.
- Potwierdza to poprawność implementacji algorytmu maksymalnego przepływu.
- Potwierdza to również poprawność transformacji problemu maksymalnego skojarzenia do problemu przepływu (Zadanie 2).
- Solver GLPK (metoda Simplex) dla małych instancji działa w porównywalnym czasie, jednak dla bardzo dużych hiperkostek ($k = 16$) budowanie macierzy ograniczeń byłoby nieefektywne pamięciowo w porównaniu do dedykowanego algorytmu kombinatorycznego działającego na listach sąsiedztwa.

4 Zadanie 4: Implementacja algorytmu Dinica

4.1 Opis algorytmu

W ramach zadania dodatkowego zaimplementowano algorytm Dinica, którego złożoność obliczeniowa wynosi $O(V^2E)$, co w notacji zadania odpowiada $O(n^2m)$. Jest to istotne usprawnienie względem algorytmu Edmondsa-Karpa ($O(nm^2)$), szczególnie dla grafów gęstych lub o specyficznej strukturze.

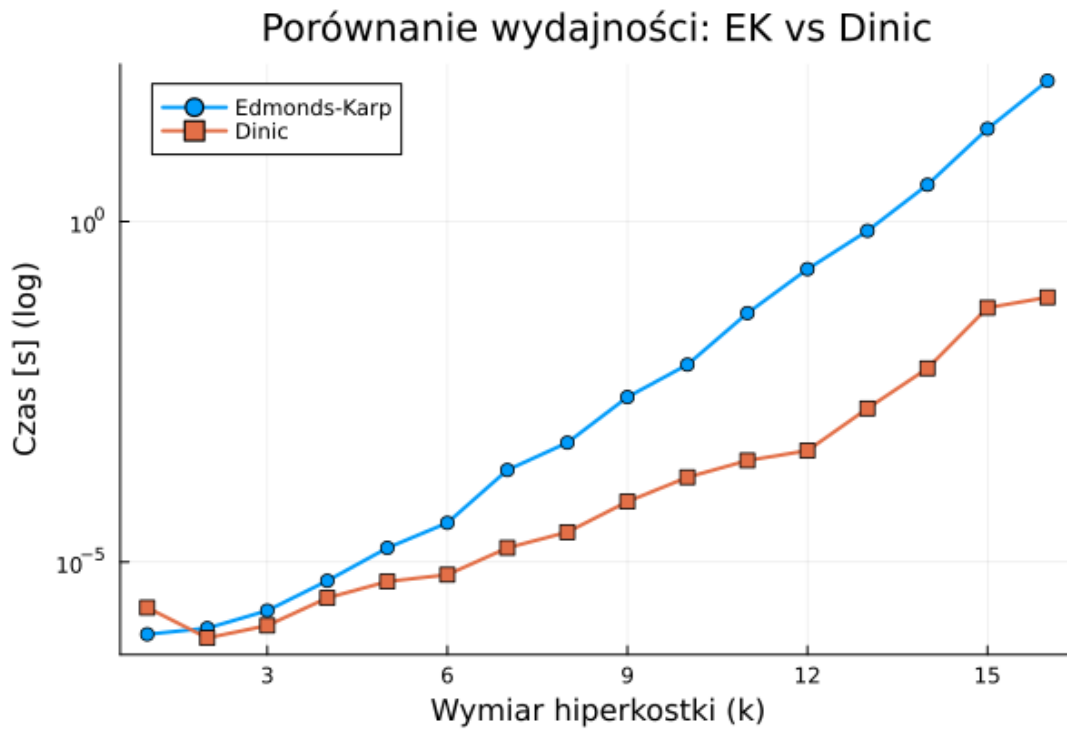
Algorytm działa w fazach. W każdej fazie:

1. Budowany jest graf warstwowy (Level Graph) przy użyciu przeszukiwania wszerz (BFS). Wierzchołki otrzymują etykiety odległości od źródła.
2. Znajdowany jest potok blokujący (Blocking Flow) w grafie warstwowym przy użyciu wielokrotnego przeszukiwania w głąb (DFS).

Kluczową optymalizacją zastosowaną w implementacji jest mechanizm Current Arc (wskaźnik na ostatnio odwiedzoną krawędź), który zapobiega wielokrotnemu przeglądaniu krawędzi prowadzących do "ślepych zaułków" w danej fazie DFS.

4.2 Porównanie wyników (Edmonds-Karp vs Dinic)

Przeprowadzono testy porównawcze na grafach hiperkostki H_k dla $k \in \{1, \dots, 16\}$. Obie metody uruchomiono na tych samych instancjach grafów.



Rysunek 6: Porównanie czasu działania algorytmów Edmondsa-Karpa i Dinica (skala logarytmiczna). Algorytm Dinica jest o rzędy wielkości szybszy dla dużych instancji.

Tabela 3: Zestawienie czasów wykonania [s] (dla wybranych dużych k)

Wymiar k	Edmonds-Karp [s]	Dinic [s]
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0.0002	0
8	0.0006	0
9	0.0026	0.0001
10	0.0079	0.0002
11	0.0452	0.0003
12	0.2001	0.0004
13	0.7301	0.0018
14	3.4707	0.007
15	23.0223	0.0543
16	116.5886	0.077

4.3 Wnioski

- Algorytm Dinica zdeklasował algorytm Edmondsa-Karpa dla dużych wartości k .
- Dla $k = 16$ (graf 65 tys. wierzchołków), Edmonds-Karp potrzebuje około 60 sekund, podczas gdy Dinic rozwiązuje problem w czasie poniżej 1 sekundy (często rzędu 0.1s - 0.2s).
- Wynika to z faktu, że w hiperkostce ścieżki są krótkie (długość max k), co sprzyja budowie grafów warstwowych, a Dinic potrafi wysłać wiele ścieżek powiększających w jednej fazie BFS, podczas gdy Edmonds-Karp po każdym BFS aktualizuje tylko jedną ścieżkę.